COP2800C Module 2 Programming Assignment

In this assignment, we will expand the functionality of the PalmerPenguins program by adding basic arithmetic operations and formatted output. This assignment introduces calculations and percentages, building on the foundational concepts from the module reading material and practice exercises.

Note: the Palmer Penguins data is included as a .csv file in the GitHub starter repository. We are not using it in this assignment but you can open it in Excel to examine the data. Note the number of rows (observations), the number of columns (variables), the data types, and range of values. This first fundamental step of familiarizing yourself with the data is known as Exploratory Data Analysis (EDA) and is a critical step in the data analysis process.

Instructions

1. **Start with Your Module 1 Code**
   Begin with the PalmerPenguins program you created in Module 1. Save the new file as PalmerPenguinsM2.java.

2. Update the 4-line ID header at the top of the file. If you change your class name as I have to PalmerPenguinsM2, then the file name must also be changed. If you retain the original class name, it does not need to change (but if you have a reference to the module number in the name as I do, you should change it).

   ```
   // PalmerPenguinsM2.java
   // Your Name
   // Submission Date
   // Program to calculate and display Palmer Penguin statistics
   ```

3. **Add New Constants**
   Below the existing species constants (SP_CHINSTRAP, etc.), add constants to represent the number of individuals observed for each species:

   ```
   static final int NUM_CHINSTRAP = 68;
   static final int NUM_GENTOO = 123;
   static final int NUM_ADELIE = 151;
   ```

4. **Calculate Total Penguins**
   Inside the main method, calculate the total number of penguins by summing the new constants. Store the result in a variable:

   ```
   int totalPenguins = NUM_CHINSTRAP + NUM_GENTOO + NUM_ADELIE;
   ```

5. **Enhance Output with Percentages**

The **System.out.printf** method in Java is used to produce formatted output, allowing you to control the appearance of text and numerical data in your program's output. The printf stands for "print formatted," and it uses a format string followed by arguments to produce well-structured output.

The format string contains placeholders known as **format specifiers** that begin with a % character. These determine how each argument will be formatted. For example:

- %d: Formats an integer.
- %.2f: Formats a floating-point number with two decimal places.
- %s: Formats a string.
- %%: Prints a literal % sign.

There are over 30 different format specifiers! Fortunately we only need a few for this assignment.

For instance, if you want to display a species name, count, and percentage, you can use System.out.printf like this:

```
System.out.printf("%s: %d (%.2f%%)\n", "Chinstrap", 68, 19.88);
```

This statement outputs:

```
Chinstrap: 68 (19.88%)
```

Using System.out.printf enhances the clarity and readability of your program's output, especially when working with numerical data and calculations. It is particularly useful in cases like this assignment, where precise percentages are displayed alongside descriptive text.

Use System.out.printf to display the percentage of each species relative to the total number of penguins, e.g.,

```
System.out.printf("%s: %d (%.2f%%)\n", SP_CHINSTRAP, NUM_CHINSTRAP,
        ((double) NUM_CHINSTRAP / totalPenguins * 100));
```

Notice how we have to perform a cast operation

(double) NUM_CHINSTRAP

in this statement. In most programming languages, when you divide two integers, the result is also an integer. **integer division** truncates (removes) any fractional part of the result. For example:

68 / 342 → 0

By **type casting** one of the integers (e.g., NUM_CHINSTRAP) to a double, Java performs **floating-point division** instead, which preserves the decimal portion of the result:

(double) NUM_CHINSTRAP / totalPenguins

68.0 / 342 → 0.1988

The multiplication by 100 then gives the "human readable" percentage:

0.1988 * 100 → 19.88%

Casting is covered in section 2.12 of our textbook

6. **Expected Output**
   Your program's output should match the following format (your percentages may vary slightly due to rounding):

   Introducing the Palmer Penguins:
   Chinstrap!
   Gentoo!
   and last but not least...
   Adelie!
   There are a total of 3 penguin species in this dataset.
   There are a total of 342 penguins in the dataset.
   Chinstrap: 68 (19.88%)
   Gentoo: 123 (35.96%)
   Adelie: 151 (44.16%)

7. **Code Readability**
   - Indent code consistently (4 spaces per level).
   - Add blank lines for readability, separating logical sections of the program.
   - Keep lines at <= 80 columns

8. **Screenshot Your Output**
   Run the program and take a screenshot of the output in the "Run I/O" pane of jGrasp. Save this screenshot to include with your submission.

Submission

- As we did in Module 1, upload the .java file and a screenshot of your program's output to your GitHub repository. Remember:
    - **Do not submit .class files!**
    - Ensure your code adheres to style guidelines.